Computational Methods for Biological Modeling and Simulation (02512/02712)
Name:Jeffrey Wang
Andrew ID: jdw2
AI use statement: Improve tricky phrasing when describing reasoning.
Reinterpreting question to better understand it. Needed it more for problems 6 and 7.
Improving latex code & formatting.

Problem Set 3

Released Thursday, October 8. Due Thursday, October 29 at 11:59pm US Eastern. Starred problems are only for 02-712 students.

Software. Problem 4 must use Python with the standard library only. Problem 1 uses CmdStanPy, and Problems 5 and 6 use cvxpy; NumPy and Matplotlib are allowed in those three problems. Parts that do not ask for code are to be done by hand, though you may use software to solve linear systems or check your work. The `PS3-starter` folder has the starting code and `START-HERE.txt`, which gives installation steps and the commands for each problem. The `PS3-data` folder has the data files. Submit your written answers, the files you wrote or modified, and your plots. The CmdStanPy installation guide, its first example, the description of Stan program blocks, and the cvxpy introduction are useful references.

Problem 1 (25 points): A laboratory follows the growth of a human cell line in culture and records the cell density C, in thousands of cells per mL, every few days. The density follows a logistic curve with plateau $K > 0$, growth rate $r > 0$ per day, and halfway day t_0,

$$C_{\text{curve}}(t) = \frac{K}{1 + \exp[-r(t - t_0)]}.$$

Measurements scatter multiplicatively around the curve, so that the natural log of each measurement equals the log of the curve plus an independent normal error with mean zero and unknown standard deviation σ. Write $a = \log K$, $\ell = \log r$, and $m(t) = a - \log(1 + \exp[-r(t - t_0)])$ for the log of the curve. Before seeing any data, the lab gives a, ℓ, and t_0 independent normal priors with means 7, -1.5, and 12 and standard deviations 3, 3, and 10. For σ it uses a half-normal prior with scale 1, which is the distribution of the absolute value of a normal draw with mean zero and standard deviation 1.

a. (3 points) Write this description as a generative model. Using the observed log densities $y_j = \log C_j$ of the readings C_j at days t_j, say which quantities are fixed inputs and which are unknown, and write the priors and the distribution of y_j given the parameters. Give the order of the random draws needed to simulate a dataset from the full model.

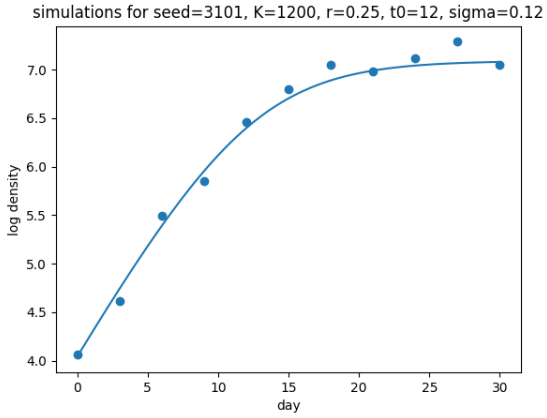
- **Fixed inputs:** the days $t_1, \ldots, t_N$, the number of readings N, and the prior means and standard deviations.
- **Observed data:** $y_j = \log C_j$ for $j = 1, \ldots, N$.
- **Unknown parameters:** a, ℓ, t_0, σ, with $K = e^a$ and $r = e^\ell$.

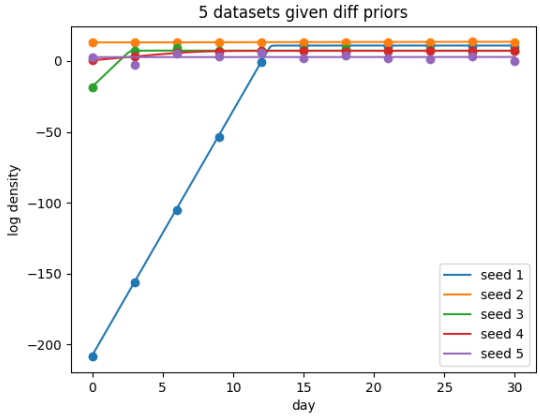
- **Priors:**
  $a \sim \text{Normal}(7, 3^2)$
  $\ell \sim \text{Normal}(-1.5, 3^2)$
  $t_0 \sim \text{Normal}(12, 10^2)$
  $\sigma \sim \text{Half-Normal}(0, 1)$
- **Distribution of y_j:** independently for each j,

$$y_j \mid a, \ell, t_0, \sigma \sim \text{Normal}\big(m(t_j), \sigma^2\big), \quad m(t_j) = a - \log\left(1 + \exp[-e^{\ell}(t_j - t_0)]\right).$$

- **Order of draws:**
  1. Draw a, ℓ, t_0, σ from their priors.
  2. Compute $r = e^{\ell}$ and $m(t_j)$ for each day.
  3. For each j, draw $y_j \sim \text{Normal}(m(t_j), \sigma^2)$.

b. (6 points) Complete the two blocks marked `P1b` in `simulate_growth.py`. The function `draw_parameters` should draw (a, r, t_0, σ) from the priors, and `simulate_densities` should draw one density for each day given the parameters. Run the script at its default settings, which fix $K = 1200$, $r = 0.25$, $t_0 = 12$, and $\sigma = 0.12$, use seed 3101, and simulate days $0, 3, \ldots, 30$. Plot log density against day together with $m(t)$. Check your code in two ways. With `--sigma 0` every point should lie on the curve. With `--repeats 1000`, report the mean and standard deviation of the log errors $y_j - m(t_j)$ and compare them with 0 and 0.12. Then simulate five datasets with `--from-prior`, using seeds 1 through 5, and plot them. Using your five draws, describe the range of growth curves these priors allow.





Check:
`--sigma 0` shows the largest gap between y_j and $m(t_j)$ is exactly $0 \rightarrow$ every point lies on the curve
`--repeats 1000` the log errors have $\mu = -0.0009 \approx 0$ and $\sigma = 0.1194 \approx 0.12$
Prior draw:

seed	K	r (per day)	t_0 (day)	σ
1	52,300	17.3	12.7	0.77
2	1,220,000	0.031	15.9	0.15
3	1,460	9.49	2.7	0.99
4	1,240	0.90	7.4	0.35
5	31.9	0.0071	18.7	2.29

The plateau ranges from about 30 to 1.2 million (thousands of cells per mL), and the rate ranges from 0.007 to 17 per day. The noise ranges from about 15% to a factor of $e^{2.29} \approx 10$. These priors allow curves like seed 4, which looks like our simulation, but they also allow many that are not biologically plausible.

c. (4 points) Fit `growth_base.stan` to your seed-3101 dataset with `fit_growth.py`. For each line of your model in (a), identify the corresponding statement in the Stan file. Report the posterior means and 90% intervals for K, r, t_0, and σ next to the values used in the simulation. The intervals are the `q05` and `q95` columns of `summary.csv`, and the rows `exp(a)` and `exp(logr)` give K and r. Did the fit recover them?

Model to Stan line-by-line:

Model (a)	`growth_base.stan`
Fixed inputs N, t_j; observed y_j	`data { int N; vector[N] t; vector[N] y; }`
Unknowns a, ℓ, t_0	`parameters { real a; real logr; real t0;`
Unknown $\sigma > 0$	`real<lower=0> sigma; }`
$a \sim \text{Normal}(7, 3^2)$	`a ~ normal(7, 3);`
$\ell \sim \text{Normal}(-1.5, 3^2)$	`logr ~ normal(-1.5, 3);`
$t_0 \sim \text{Normal}(12, 10^2)$	`t0 ~ normal(12, 10);`
$\sigma \sim \text{Half-Normal}(0, 1)$	`sigma ~ normal(0, 1);` with `<lower=0>`
$m(t_j) = a - \log(1 + e^{-r(t_j - t_0)})$	`real m = a - log1p_exp(-exp(logr) * (t[n] - t0));`
$y_j \sim \text{Normal}(m(t_j), \sigma^2)$	`y[n] ~ normal(m, sigma);`

Posterior vs. true values:

Parameter	Posterior mean	90% interval	True value
K	1324	(1188, 1488)	1200
r	0.251	(0.221, 0.279)	0.25
t_0	12.50	(11.24, 13.99)	12
σ	0.123	(0.079, 0.193)	0.12

Yes, the fit recovered them because every true value lies inside its 90% interval, some of which are almost the exact same value.

d. (7 points) The file `growth.json` has measurements from twelve cell lines, in the parallel lists `cell_line` (1 through 12), `day`, and `density`. Extend the model so that each cell line i has its own log plateau a_i and log rate ℓ_i. Across cell lines, the a_i are normal with unknown mean μ_a and standard deviation τ_a, and the ℓ_i are normal with unknown mean μ_ℓ and standard deviation τ_ℓ, all independent given these population parameters. The halfway day t_0 and measurement standard deviation σ are shared by all cell lines and keep their priors. Give μ_a and μ_ℓ the priors used for

a and ℓ before, and give τ_a and τ_ℓ independent half-normal priors with scale 1. In Stan, keep the names `a`, `logr`, `t0`, and `sigma`, and call the four new parameters `mu_a`, `mu_logr`, `tau_a`, and `tau_logr`.

Write the extended generative model. Then edit the blocks marked `P1d` in `growth_extend.stan`, which starts as a copy of the base model with the data block already extended. Run it on `growth.json` and report means and 90% intervals for e^{μ_a}, e^{μ_ℓ}, τ_a, t_0, and σ (the rows `exp(mu_a)` and `exp(mu_logr)` give the first two). Explain what σ and τ_a each describe.

Extended generative model: We still know the days t_j and cell line $c_j \in \{1, \dots, 12\}$ for each reading j. Now the unknowns are $\mu_a, \mu_\ell, \tau_a, \tau_\ell, \{a_1, \dots, a_{12}\}, \{\ell_1, \dots, \ell_{12}\}, t_0, \sigma$.

$$
\begin{aligned}
&\mu_a \sim \text{Normal}(7, 3^2), \quad \mu_\ell \sim \text{Normal}(-1.5, 3^2), \quad \tau_a, \tau_\ell \sim \text{Half-Normal}(0, 1), \\
&t_0 \sim \text{Normal}(12, 10^2), \quad \sigma \sim \text{Half-Normal}(0, 1), \\
&a_i \mid \mu_a, \tau_a \sim \text{Normal}(\mu_a, \tau_a^2), \quad \ell_i \mid \mu_\ell, \tau_\ell \sim \text{Normal}(\mu_\ell, \tau_\ell^2), \quad i = 1, \dots, 12, \\
&y_j \mid \cdot \sim \text{Normal}\big(m_j, \sigma^2\big), \quad \text{where } m_j = a_{c_j} - \log\left(1 + \exp[-e^{\ell_{c_j}}(t_j - t_0)]\right).
\end{aligned}
$$

Draw the population parameters, t_0 and σ, then each a_i and ℓ_i, then each y_j.
Results:

	Mean	90% interval
e^{μ_a}	1186	(993, 1405)
e^{μ_ℓ}	0.220	(0.181, 0.264)
τ_a	0.344	(0.233, 0.504)
t_0	11.94	(11.55, 12.35)
σ	0.121	(0.108, 0.136)

σ is the measurement noise within a cell line on the log scale. $\sigma = 0.121$ means readings are off by about 12.1%.
τ_a is the variation between cell lines. $\tau_a = 0.344$ means a typical line's plateau is within a factor of about $e^{0.344} \approx 1.41$ of the population plateau e^{μ_a}.

e. (5 points) Cell line 12 has only four readings, all taken by day 9. Compare the plateau intervals for cell lines 12 and 1 in your extended fit. Where does the extended fit get its information about cell line 12's plateau, and what assumptions does that information depend on? If cell line 12 were fit on its own with the base model, would its plateau be unidentified, or only weakly informed by its four readings? Explain using the shape of the curve before day 9.

Cell line	Mean K_i	90% interval
1	1474	(1369, 1586)
12	1573	(1283, 1895)

Line 12's interval is about 3 times wider than line 1's (612 vs. 217) because line 1 has readings on its plateau (days 18–30), while line 12 stops at day 9, before the halfway day.
Where the information comes from: the other eleven lines. They estimate μ_a and τ_a, which act as a prior for a_{12}, and they pin down the shared $t_0 \approx 11.9$. With t_0 known, the four early

readings tell how far below the plateau the curve still is. This assumes line 12 comes from the same population as the other lines, shares their t_0 and σ, and keeps following the logistic curve after day 9.

Line 12 only: weakly informed because before t_0, $\log C \approx (a - rt_0) + rt$ is nearly a straight line, so the readings fix the slope and intercept but not a and t_0 separately. A larger K with a later t_0 fits almost as well. The curve bends slightly as it nears t_0, so the data still say a little about K. Fitting line 12 alone gives K in (415, 5524), much narrower than the prior but far wider than the pooled interval.

Problem 2 (15 points): In this problem we apply the zero-finding methods from class to a hypothetical molecular design question. A charged nanoparticle sits inside a synthetic nanopore whose two ends carry fixed surface charges, and we treat the pore as a one-dimensional channel in which the particle can move. The particle has effective charge $+1$ in arbitrary units. The left mouth of the pore, at $x = -1.5$, carries charge $+2$, and the right mouth, at $x = 2$, carries charge $+1.5$, so both mouths repel the particle. A simple model for the potential energy at axial position x is

$$E(x) = \frac{2}{x + 1.5} - \frac{1.5}{x - 2}.$$

The singularities at the two mouths lie outside the interval $[-1, 1]$ studied below, so we ignore them. We want to estimate the minimum-energy position of the particle by finding a zero of $F(x) = E'(x)$.

Bisection starts from an interval where F has opposite signs at the two endpoints, evaluates F at the midpoint, and retains the half whose endpoints still differ in sign. The secant method needs no derivative and keeps no interval. From the two most recent estimates it takes the zero of the line through $(x_{k-1}, F(x_{k-1}))$ and $(x_k, F(x_k))$,

$$x_{k+1} = x_k - F(x_k)\frac{x_k - x_{k-1}}{F(x_k) - F(x_{k-1})}.$$

Newton–Raphson uses the tangent line instead, $x_{k+1} = x_k - F(x_k)/F'(x_k)$. For each method below, report the final estimate $\hat{x}$ and two error measures. The forward error $|\hat{x} - x^*|$ is the distance from the true zero x^*. For this exercise, report a bound or estimate without solving for x^* exactly. For bisection, use the largest distance from $\hat{x}$ to a point of the interval it was computed from, and for the secant and Newton–Raphson methods, use the size of the last update. The backward error $|F(\hat{x})|$ is the amount by which F would have to be shifted for $\hat{x}$ to be an exact zero.

a. (2 points) Derive $F(x) = E'(x)$, and explain why the minimum-energy position must be a zero of F.

$F(x) = E'(x) = -\frac{2}{(x+1.5)^2} + \frac{1.5}{(x-2)^2}$
$E \to +\infty$ at both mouths, so it has a minimum inside $(-1.5, 2)$, and at an interior minimum the slope is zero. Since $E''(x) = \frac{4}{(x+1.5)^3} - \frac{3}{(x-2)^3} > 0$ on $(-1.5, 2)$, F is increasing and this zero is the only one.

b. (4 points) The particle must lie in $[-1, 1]$. Perform three iterations of the bisection method on this interval, showing your work. The final estimate is the third midpoint you compute, not the midpoint of the interval that remains.

Start with $[a, b] = [-1, 1]$, where $F(-1) = -8 + \frac{1.5}{9} = -7.833 < 0$ and $F(1) = -\frac{2}{6.25} + 1.5 = 1.18 > 0$.

Iter.	a	b	midpoint m	$F(m)$	sign	new interval
1	-1	1	0	$-\frac{2}{2.25}+\frac{1.5}{4}=-\frac{37}{72}=-0.5139$	$-$	$[0,1]$
2	0	1	0.5	$-\frac{2}{4}+\frac{1.5}{2.25}=\frac{1}{6}=0.1667$	$+$	$[0,0.5]$
3	0	0.5	0.25	$-\frac{2}{3.0625}+\frac{1.5}{3.0625}=-\frac{8}{49}=-0.1633$	$-$	$[0.25,0.5]$

$\hat{x}=0.25$
Forward error: $|\hat{x}-x^*|\le 0.25$ (farthest point of $[0,0.5]$)
Backward error: $|F(\hat{x})|=8/49\approx 0.163$

c. (3 points) Perform three iterations of the secant method starting from $x_0=-1$ and $x_1=1$, showing your work.

Using $x_{k+1}=x_k-F(x_k)\dfrac{x_k-x_{k-1}}{F(x_k)-F(x_{k-1})}$, with $F(x_0)=F(-1)=-7.8333$ and $F(x_1)=F(1)=1.18$:

$$x_2=1-1.18\cdot\frac{1-(-1)}{1.18-(-7.8333)}=1-\frac{2.36}{9.0133}=\frac{499}{676}=0.73817, \qquad F(x_2)=0.54283,$$
$$x_3=0.73817-0.54283\cdot\frac{0.73817-1}{0.54283-1.18}=0.73817-0.22306=0.51510, \qquad F(x_3)=0.18776,$$
$$x_4=0.51510-0.18776\cdot\frac{0.51510-0.73817}{0.18776-0.54283}=0.51510-0.11796=0.39714, \qquad F(x_4)=0.02816.$$

$\hat{x}=x_4=0.3971$
Forward error: $|x_4-x_3|\approx 0.118$
Backward error: $|F(\hat{x})|\approx 0.0282$

d. (4 points) Starting from $x_0=0$, perform three iterations of the Newton–Raphson method, showing your work.

$F'(x)=\dfrac{4}{(x+1.5)^3}-\dfrac{3}{(x-2)^3}$, $x_{k+1}=x_k-F(x_k)/F'(x_k)$

k	x_k	$F(x_k)$	$F'(x_k)$	update $-F/F'$	x_{k+1}
0	0	-0.513889	1.560185	$+0.329377$	0.329377
1	0.329377	-0.060173	1.296764	$+0.046402$	0.375779
2	0.375779	1.759×10^{-4}	1.306201	-1.347×10^{-4}	0.375644

$\hat{x}=x_3=0.375644$
Forward error: $|x_3-x_2|\approx 1.35\times10^{-4}$
Backward error: $|F(\hat{x})|\approx 2.9\times10^{-9}$

e. (2 points) Evaluate E at each final estimate. Rank the methods by each error measure and by energy.

Method	$\hat{x}$	Forward error	Backward error $\lvert F(\hat{x})\rvert$	$E(\hat{x})$
Bisection	0.25	≤ 0.25	0.163	$\frac{2}{1.75}+\frac{1.5}{1.75}=2.000000$
Secant	0.397143	≈ 0.118	0.0282	1.990046
Newton–Raphson	0.375644	$\approx 1.35\times10^{-4}$	2.9×10^{-9}	1.989743

Forward error, backward error, and energy all rank the methods the same, best to worst: Newton–Raphson, secant, bisection. The energies are much closer than the positions because E is flat near its minimum.

Problem 3 (10 points): We have been monitoring new COVID cases at two-week intervals and need estimates at a finer time resolution, to test how specific events might have contributed to the spread. The measurements are

t (weeks)	0	2	4	6
new cases C (thousands)	0.5	1.7	3.5	10.1

and we want estimates for weeks 1, 3, and 5. For each method below, give the interpolating function or functions on $[0, 6]$ and the values they predict at $t = 1$, 3, and 5.

a. (2 points) Piecewise linear interpolation.

Slopes are $\frac{1.7-0.5}{2} = 0.6$, $\frac{3.5-1.7}{2} = 0.9$, and $\frac{10.1-3.5}{2} = 3.3$:

$$L(t) = \begin{cases} 0.5 + 0.6\,t, & 0 \le t \le 2, \\ 1.7 + 0.9\,(t-2), & 2 \le t \le 4, \\ 3.5 + 3.3\,(t-4), & 4 \le t \le 6. \end{cases}$$

$L(1) = 1.1$, $L(3) = 2.6$, $L(5) = 6.8$

b. (4 points) A single cubic polynomial through all four points.

Using Newton's divided differences:

t	C	1st	2nd	3rd
0	0.5			
		$\frac{1.7-0.5}{2} = 0.6$		
2	1.7		$\frac{0.9-0.6}{4} = 0.075$	
		$\frac{3.5-1.7}{2} = 0.9$		$\frac{0.6-0.075}{6} = 0.0875$
4	3.5		$\frac{3.3-0.9}{4} = 0.6$	
		$\frac{10.1-3.5}{2} = 3.3$		
6	10.1			

$$\begin{aligned} p(t) &= 0.5 + 0.6\,t + 0.075\,t(t-2) + 0.0875\,t(t-2)(t-4) \\ &= 0.0875\,t^3 - 0.45\,t^2 + 1.15\,t + 0.5. \end{aligned}$$

$p(1) = 1.2875$, $p(3) = 2.2625$, $p(5) = 5.9375$

c. (4 points) A quadratic spline through every data point, with matching first derivatives at $t = 2$ and $t = 4$ and with $C'(0) = 0$ (cases were initially stable). Write the linear system for the nine coefficients and solve it, by hand or with code.

On each interval $[t_i, t_{i+1}]$ with $t_1 = 0$, $t_2 = 2$, $t_3 = 4$, write $S_i(t) = a_i + b_i(t - t_i) + c_i(t - t_i)^2$, so $S_i'(t) = b_i + 2c_i(t - t_i)$. The nine conditions are

$$
\begin{aligned}
&S_1(0) = a_1 = 0.5, && S_1(2) = a_1 + 2b_1 + 4c_1 = 1.7,\\
&S_2(2) = a_2 = 1.7, && S_2(4) = a_2 + 2b_2 + 4c_2 = 3.5,\\
&S_3(4) = a_3 = 3.5, && S_3(6) = a_3 + 2b_3 + 4c_3 = 10.1,\\
&S_1'(2) = S_2'(2):\ b_1 + 4c_1 - b_2 = 0, && S_2'(4) = S_3'(4):\ b_2 + 4c_2 - b_3 = 0,\\
&S_1'(0) = b_1 = 0.
\end{aligned}
$$

$$
\begin{pmatrix}
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0\\
1 & 2 & 4 & 0 & 0 & 0 & 0 & 0 & 0\\
0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0\\
0 & 0 & 0 & 1 & 2 & 4 & 0 & 0 & 0\\
0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0\\
0 & 0 & 0 & 0 & 0 & 0 & 1 & 2 & 4\\
0 & 1 & 4 & 0 & -1 & 0 & 0 & 0 & 0\\
0 & 0 & 0 & 0 & 1 & 4 & 0 & -1 & 0\\
0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0
\end{pmatrix}
\begin{pmatrix} a_1\\ b_1\\ c_1\\ a_2\\ b_2\\ c_2\\ a_3\\ b_3\\ c_3 \end{pmatrix}
=
\begin{pmatrix} 0.5\\ 1.7\\ 1.7\\ 3.5\\ 3.5\\ 10.1\\ 0\\ 0\\ 0 \end{pmatrix}.
$$

Solving by hand...
$a_1 = 0.5$, $b_1 = 0$, $c_1 = 1.2/4 = 0.3$
$a_2 = 1.7$, $b_2 = b_1 + 4c_1 = 1.2$, $c_2 = (3.5 - 1.7 - 2.4)/4 = -0.15$
$a_3 = 3.5$, $b_3 = b_2 + 4c_2 = 0.6$, $c_3 = (10.1 - 3.5 - 1.2)/4 = 1.35$

$$
S(t) = \begin{cases}
0.5 + 0.3\,t^2, & 0 \le t \le 2,\\
1.7 + 1.2\,(t-2) - 0.15\,(t-2)^2, & 2 \le t \le 4,\\
3.5 + 0.6\,(t-4) + 1.35\,(t-4)^2, & 4 \le t \le 6.
\end{cases}
$$

$S(1) = 0.8$, $S(3) = 2.75$, $S(5) = 5.45$

Problem 4 (20 points): In this programming problem you will fit a recovery-to-plateau model to noisy data. The model is used in fluorescence recovery after photobleaching (FRAP) and in other kinetic experiments. In FRAP, molecules are labeled with a fluorescent probe, and the probe is then deliberately destroyed within a small region of a cell. The mean fluorescence in that region recovers toward an asymptote as mobile molecules exchange with the bleached ones. A common approximation for the normalized mean recovery is

$$
R(t) = R_\infty \left(1 - e^{-kt}\right),
$$

where $R_\infty \in (0, 1]$ is the plateau and $k > 0$ is an effective rate constant. Given observations (t_i, R_i), we estimate (R_∞, k) by minimizing the sum of squared errors

$$
\Phi(R_\infty, k) = \sum_{i=1}^{n} \left(R_i - R_\infty \left(1 - e^{-kt_i}\right)\right)^2.
$$

a. (3 points) Derive the gradient of Φ, giving closed-form expressions for $\partial\Phi/\partial R_\infty$ and $\partial\Phi/\partial k$.

$$\frac{\partial \Phi}{\partial R_\infty} = -2 \sum_{i=1}^{n} \left(R_i - R_\infty (1 - e^{-k t_i}) \right) \left(1 - e^{-k t_i} \right),$$
$$\frac{\partial \Phi}{\partial k} = -2 R_\infty \sum_{i=1}^{n} \left(R_i - R_\infty (1 - e^{-k t_i}) \right) t_i e^{-k t_i}.$$

b. (4 points) Rather than derive the analytic Hessian, we approximate it by finite differences. Give forward-difference formulas, using values of Φ only, for the three distinct Hessian entries, with steps ΔR in R_∞ and Δk in k. Applying a forward difference twice gives, for a function f of one variable, $f''(x) \approx [f(x+2h) - 2f(x+h) + f(x)]/h^2$; the mixed entry follows from taking a forward difference in R_∞ of a forward difference in k.

Given that $R = R_\infty$ we will use the following finite difference approximations for the Hessian entries:

$$\frac{\partial^2 \Phi}{\partial R^2} \approx \frac{\Phi(R + 2\Delta R, k) - 2\Phi(R + \Delta R, k) + \Phi(R, k)}{\Delta R^2},$$
$$\frac{\partial^2 \Phi}{\partial k^2} \approx \frac{\Phi(R, k + 2\Delta k) - 2\Phi(R, k + \Delta k) + \Phi(R, k)}{\Delta k^2},$$
$$\frac{\partial^2 \Phi}{\partial R\, \partial k} \approx \frac{\Phi(R + \Delta R, k + \Delta k) - \Phi(R + \Delta R, k) - \Phi(R, k + \Delta k) + \Phi(R, k)}{\Delta R\, \Delta k}.$$

The Hessian is symmetric, so $\partial^2 \Phi / \partial k\, \partial R$ is the same entry.

c. (6 points) Give pseudocode for a Newton–Raphson solver that takes a starting point $(R_{\infty,0}, k_0)$, steps $(\Delta R, \Delta k)$, an iteration count q, and data vectors $\vec{t}$ and $\vec{R}$, and returns (R_∞, k) after q iterations. Use the gradient from (a) and the Hessian from (b).

```
function NEWTON(R0, k0, dR, dk, q, t, R):
    (x, y) = (R0, k0)
    repeat q times:
        gx, gy = GRADIENT(x, y, t, R)
        f00 = PHI(x, y)
        Hxx = (PHI(x+2dR, y) - 2 PHI(x+dR, y) + f00) / dR^2
        Hyy = (PHI(x, y+2dk) - 2 PHI(x, y+dk) + f00) / dk^2
        Hxy = (PHI(x+dR, y+dk) - PHI(x+dR, y) - PHI(x, y+dk) + f00) / (dR dk)

        det = Hxx Hyy - Hxy^2
        sx = (-gx Hyy + gy Hxy) / det
        sy = (-gy Hxx + gx Hxy) / det
        (x, y) = (x + sx, y + sy)
    return (x, y)
```

d. (4 points) Implement your solver in Python with the standard library only. The program should take the name of an input file as its command-line argument and read a file of the form

```
R_inf0
k0
Delta_R
Delta_k
q
n
t_1  R_1
...
t_n  R_n
```

and print the final estimates `R_inf k` on one line, separated by a space.

Run `python frap.py PS3-data/frap_a.txt`

e. (3 points) Run your solver on `frap_a.txt`, `frap_b.txt`, and `frap_c.txt` in `PS3-data` and report the estimates.

	$(R_{\infty,0}, k_0)$	q	n	$\hat{R}_\infty$	$\hat{k}$	Φ
`frap_a.txt`	$(0.7, 0.2)$	20	10	0.848	0.502	0.000285
`frap_b.txt`	$(0.55, 0.6)$	25	18	0.621	0.750	0.000221
`frap_c.txt`	$(0.85, 0.25)$	35	16	0.944	0.280	0.000295

Problem 5 (15 points): In this problem we use linear programming to design a genomics experiment. We want to sequence some tissue and can choose a combination of bulk, single-cell, and spatial sequencing, in amounts x_1, x_2, and x_3. The technologies differ in how useful their outputs are and in what they cost. We judge one unit of bulk, single-cell, and spatial sequencing to be worth 1, 2, and 3, respectively, so we want to

$$\max \; x_1 + 2x_2 + 3x_3.$$

Bulk and single-cell sequencing use tissue, of which we have 3 units. One unit of bulk needs 1 unit of tissue, and one unit of single-cell needs 2, so

$$x_1 + 2x_2 \leq 3.$$

Bulk sequencing is cheap enough that we ignore its cost, but each unit of single-cell or spatial sequencing costs 1 unit of our budget, which is 8 units, so

$$x_2 + x_3 \leq 8.$$

Finally, we have a single-cell sequencer in our lab but must send bulk and spatial samples to a company, which takes time. Bulk takes 2 units of time per unit and spatial takes 1, and we need the data back within 10 units of time, so

$$2x_1 + x_3 \leq 10.$$

We cannot do a negative amount of sequencing, so $x_1, x_2, x_3 \geq 0$.

A linear program in standard form minimizes $c^{\mathsf{T}}x$ subject to $Ax = b$ and $x \geq 0$. Each constraint $a^{\mathsf{T}}x \leq \beta$ becomes an equality by adding a slack variable $s = \beta - a^{\mathsf{T}}x \geq 0$, the unused amount of that resource (here s_1, s_2, and s_3 for tissue, budget, and time), and a maximization becomes a minimization by negating the objective. The simplex method starts at a feasible vertex, here $x = 0$ with every slack equal to the right-hand side of its constraint, and writes the slacks and the objective in terms of the variables that are zero. At each step it chooses a zero variable with a negative objective coefficient, called the entering variable (for example, the one with the most negative coefficient), and increases it until another variable, called the leaving variable, reaches zero. It then solves the leaving variable's equation for the entering variable and substitutes the result into the other equations and the objective. When no objective coefficient is negative, the current vertex is optimal.

a. (2 points) Convert the linear program to standard form.

$x = (x_1, x_2, x_3, s_1, s_2, s_3)$ with s_1, s_2, and s_3 the slack variables for tissue, budget, and time, respectively. The linear program becomes...

$$\min\ -x_1 - 2x_2 - 3x_3 \quad \text{subject to} \quad \begin{aligned} x_1 + 2x_2 + s_1 &= 3, \\ x_2 + x_3 + s_2 &= 8, \\ 2x_1 + x_3 + s_3 &= 10, \end{aligned} \qquad x_1, x_2, x_3, s_1, s_2, s_3 \geq 0.$$

That is, $c = (-1, -2, -3, 0, 0, 0)^{\mathsf{T}}$, $b = (3, 8, 10)^{\mathsf{T}}$, and $A = [\, A_x \mid I \,] = \left(\begin{array}{ccc|ccc} 1 & 2 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 2 & 0 & 1 & 0 & 0 & 1 \end{array}\right)$.

b. (7 points) Solve the linear program by the simplex method, showing your work, to find the optimal amount of each kind of sequencing.

1. $x = 0$ and minimizing the objective $z = -x_1 - 2x_2 - 3x_3$:

   $s_1 = 3 - x_1 - 2x_2, \qquad s_2 = 8 - x_2 - x_3, \qquad s_3 = 10 - 2x_1 - x_3, \qquad z = -x_1 - 2x_2 - 3x_3 = 0.$

2. x_3 enters (most negative).
   Limits: s_2: 8, s_3: 10, so s_2 leaves.
   Substituting $x_3 = 8 - x_2 - s_2$,

   $x_3 = 8 - x_2 - s_2, \qquad s_1 = 3 - x_1 - 2x_2, \qquad s_3 = 2 - 2x_1 + x_2 + s_2, \qquad z = -24 - x_1 + x_2 + 3s_2.$

3. x_1 enters (only negative).
   Limits: s_1: 3, s_3: $2/2 = 1$, so s_3 leaves.
   Substituting $x_1 = 1 + \frac{1}{2}x_2 + \frac{1}{2}s_2 - \frac{1}{2}s_3$,

   $$\begin{aligned} x_1 &= 1 + \tfrac{1}{2}x_2 + \tfrac{1}{2}s_2 - \tfrac{1}{2}s_3, & x_3 &= 8 - x_2 - s_2, \\ s_1 &= 2 - \tfrac{5}{2}x_2 - \tfrac{1}{2}s_2 + \tfrac{1}{2}s_3, & z &= -25 + \tfrac{1}{2}x_2 + \tfrac{5}{2}s_2 + \tfrac{1}{2}s_3. \end{aligned}$$

4. All objective coefficients are nonnegative, so this vertex is optimal:

   $$x_1 = 1 \text{ (bulk)}, \qquad x_2 = 0 \text{ (single-cell)}, \qquad x_3 = 8 \text{ (spatial)}, \qquad s_1 = 2,$$

   with maximum value $x_1 + 2x_2 + 3x_3 = 1 + 24 = 25$.

c. (2 points) One of the three resource constraints is not tight at the optimum. Which resource is in excess, and how much is left over?

Tissue is in excess. The plan uses $x_1 + 2x_2 = 1$ of 3 units, so $s_1 = 2$ units are left over.

d. (4 points) Complete the block marked `P5d` in `sequencing.py` with this linear program, writing the three resource constraints as a Python list with the function's arguments `tissue`, `budget`, and `time` as their limits, and run it. The script solves the program and then solves it again with one more unit of each resource in turn. Report the optimum and the dual values of the three resource constraints. Compare the dual values with the coefficients of the slack variables in the objective at your final simplex step, and with the gains in value from the extra units. Explain the dual value for the resource you identified in (c).

(tissue, budget, time)	x	Optimum	Gains
$(3, 8, 10)$	$(1, 0, 8)$	25	–
$(4, 8, 10)$	$(1, 0, 8)$	25	0
$(3, 9, 10)$	$(0.5, 0, 9)$	27.5	2.5
$(3, 8, 11)$	$(1.5, 0, 8)$	25.5	0.5

Optimum: $x = (1, 0, 8)$ with value 25, same as (b).
Dual values (tissue, budget, time): $(0, 2.5, 0.5)$. These equal the coefficients of s_1, s_2, s_3 in the final simplex objective $z = -25 + \frac{1}{2}x_2 + \frac{5}{2}s_2 + \frac{1}{2}s_3$ (s_1 is absent, so 0) and the gains from one extra unit of each resource.
Tissue's dual value is 0 because 2 units are already unused, so an extra unit cannot change the optimum.

Problem 6 (15 points): Genomic deconvolution explains gene expression measured in a bulk tissue sample as a mixture of contributions from different cell types. We measure n genes in a bulk blood sample, giving a vector $\vec{v} = (v_1, \ldots, v_n)$. We also have reference expression for k cell types measured separately on the same genes, giving an $n \times k$ matrix R whose column j is the expression signature of cell type j. We want coefficients $\vec{c} = (c_1, \ldots, c_k)$ giving the amount of each cell type, so that $R\vec{c}$ explains $\vec{v}$. Assume $n \gg k$ and that the columns of R are linearly independent. In all parts, allow $\vec{c}$ to range over $\mathbb{R}^k$, without imposing nonnegativity or a sum-to-one constraint.

a. (2 points) We first want the least-squares fit, the $\vec{c}$ minimizing $\|R\vec{c} - \vec{v}\|^2$. Show how to find it by solving a linear system.

$\nabla f = 2R^\mathsf{T}R\vec{c} - 2R^\mathsf{T}\vec{v} = 0 \to R^\mathsf{T}R\vec{c} = R^\mathsf{T}\vec{v}$
These are the normal equations for the least-squares problem. The solution is then unique because the columns of R are independent, so $R^\mathsf{T}R$ is invertible and $\vec{c} = (R^\mathsf{T}R)^{-1}R^\mathsf{T}\vec{v}$ (i.e. there's a bottom).

b. (4 points) A least-squares fit tends to assign small amounts, positive or negative, to every cell type, even to types that are absent, by fitting the noise. One remedy adds a penalty on the sizes of the coefficients. A variant that can be posed as a linear program measures both the fit and the penalty with absolute values:

$$\min_{\vec{c}} \sum_{i=1}^{n} |(R\vec{c})_i - v_i| + \lambda \sum_{j=1}^{k} |c_j|$$

for a constant $\lambda \geq 0$. Show how to pose this as a linear program. (Hint: define auxiliary variables, and note that $x \geq |y|$ if and only if $x \geq y$ and $x \geq -y$.)

Let e_i stand in for $|(R\vec{c})_i - v_i|$ and u_j for $|c_j|$:

$$\begin{aligned} \min_{\vec{c},\vec{e},\vec{u}} \quad & \sum_{i=1}^{n} e_i + \lambda \sum_{j=1}^{k} u_j \\ \text{subject to} \quad & e_i \geq (R\vec{c})_i - v_i, \quad e_i \geq v_i - (R\vec{c})_i, \quad i = 1, \ldots, n, \\ & u_j \geq c_j, \quad u_j \geq -c_j, \quad j = 1, \ldots, k. \end{aligned}$$

Everything is linear, so this is an LP. Minimizing pushes each e_i and u_j down to its absolute value, so the optimal $\vec{c}$ is the same.

c. (3 points) It is more common to keep squared error for the fit and absolute values for the penalty:

$$\min_{\vec{c}} \ \|R\vec{c} - \vec{v}\|^2 + \lambda \sum_{j=1}^{k} |c_j|.$$

Pose this as a quadratic program, and show that it is convex by computing the Hessian of the quadratic program's objective with respect to all of its variables.

Let u_j stand in for $|c_j|$:

$$\begin{aligned} \min_{\vec{c},\vec{u}} \quad & \|R\vec{c} - \vec{v}\|^2 + \lambda \sum_{j=1}^{k} u_j \\ & = \vec{c}^{\,\mathsf{T}} R^{\mathsf{T}} R\, \vec{c} - 2\vec{v}^{\,\mathsf{T}} R\, \vec{c} + \vec{v}^{\,\mathsf{T}} \vec{v} + \lambda\, \mathbf{1}^{\mathsf{T}} \vec{u} \\ \text{subject to} \quad & u_j \geq c_j, \quad u_j \geq -c_j, \quad j = 1, \ldots, k. \end{aligned}$$

This is a quadratic objective with linear constraints, so this is a QP. The gradient is $(2R^{\mathsf{T}}R\,\vec{c} - 2R^{\mathsf{T}}\vec{v},\ \lambda\mathbf{1})$, so the Hessian over $(\vec{c}, \vec{u})$ is

$$H = \begin{pmatrix} 2R^{\mathsf{T}}R & 0 \\ 0 & 0 \end{pmatrix} \in \mathbb{R}^{2k \times 2k}.$$

$(\vec{x}, \vec{y})^{\mathsf{T}} H (\vec{x}, \vec{y}) = 2\|R\vec{x}\|^2 \geq 0$, so H is positive semidefinite and the QP is convex.

d. (6 points) Complete the three functions marked `P6d` in `deconvolution.py`: the least-squares solution from (a), and the programs from (b) and (c), written in cvxpy with their auxiliary variables. The script calls the program from (b) the absolute-error fit and the program from (c) the penalized fit. It fits sample A by least squares and by the penalized fit with $\lambda = 10$, 30, and 100, and fits sample B by least squares and by the absolute-error fit with $\lambda = 0$. Run the script on `deconvolution.json`, which has the signatures of six cell types (monocytes, B cells, CD4 T cells, CD8 T cells, NK cells, and neutrophils) on 50 genes and two bulk samples. Sample A came from a donor whose NK cells and neutrophils were depleted before the measurement. What amounts does least squares assign to these two types in sample A, and what does increasing λ do to them and to the other types? For sample B, compare the least-squares and absolute-error fits and the genes with the largest residuals (predicted minus measured expression). What do you think happened to some of the measurements, and why does the absolute-error fit respond less to them?

	Monocytes	B cells	CD4 T cells	CD8 T cells	NK cells	Neutrophils	Total
A (least squares)	0.366	0.181	0.310	0.086	−0.064	0.011	0.890
A (penalized, $\lambda = 10$)	0.357	0.173	0.300	0.083	−0.012	0.001	0.901
A (penalized, $\lambda = 30$)	0.350	0.166	0.299	0.075	0	0	0.891
A (penalized, $\lambda = 100$)	0.335	0.145	0.310	0.049	0	0	0.839
B (least squares)	0.176	0.415	0.659	−0.084	−0.104	0.002	1.064
B (absolute error, $\lambda = 0$)	0.124	0.265	0.309	0.168	0.179	0.083	1.127

Sample A: Least squares results in NK -0.064 and neutrophils 0.011. Increasing λ shrinks both to exactly 0 by $\lambda = 30$. The other types shrink only slightly until $\lambda = 100$, where B cells $0.181 \to 0.145$ and CD8 T from $0.086 \to 0.049$.
Sample B: Least squares has two huge residuals, g20 (-27.2) and g12 (-7.3), while every other gene is within ~ 4. g20 is a CD4 T marker and g12 is a B cell marker, so least squares inflates CD4 T (0.659) and B cells (0.415) and pushes CD8 T and NK negative. The absolute-error fit gives all positive amounts and leaves g20 and g12 with even larger residuals (-29.6, -9.1), close to a least-squares fit with those two genes dropped.
I think there might've been issues with the g20 and g12 measurements like contamination because their reads are much higher than the rest. Squared error (r^2) means an outlier can outweigh many normal genes. Absolute error ($|r|$) means each gene's pull on the fit is bounded and the outliers matter less so the absolute-error fit responds less to them.

***Problem 7 (15 points):** The proof that Newton–Raphson approximately squares its error on each step assumes $f'(x_i) \neq 0$. That assumption becomes a problem if the zero x^* of f is also a zero of the derivative, $f'(x^*) = 0$. When $f'(x_i) \neq 0$,

$$x^* - x_{i+1} = -\frac{f''(\xi)}{2f'(x_i)}(x^* - x_i)^2$$

for some ξ between x^* and x_i. If $f'(x_i) \to 0$ as $x_i \to x^*$, this does not obviously imply convergence, since $f''(\xi)/(2f'(x_i))$ could grow faster than $x^* - x_i$ shrinks. Here we show that the method still converges under the conditions below when $f'(x^*) = 0$ and find its rate.

a. (6 points) Suppose $f'(x^*) = 0$ and f'' is nonzero between x^* and x_i. Show that

$$x^* - x_{i+1} = \frac{f''(\xi_1)}{2f''(\xi_2)}(x^* - x_i)$$

for some ξ_1, ξ_2 between x^* and x_i. (Hint: recall the mean value theorem. If a function g is differentiable between a and b and continuous at both, then $g(b) - g(a) = g'(\xi)(b - a)$ for some ξ between a and b. Apply it to f' on the interval between x^* and x_i.)

Assume $x_i \neq x^*$. By the mean value theorem on f', for some ξ_2 between x^* and x_i,

$$f'(x_i) - f'(x^*) = f''(\xi_2)(x_i - x^*).$$

Since $f'(x^*) = 0$, $f'(x_i) = -f''(\xi_2)(x^* - x_i) \neq 0$, so the given error formula holds with ξ_1:

$$x^* - x_{i+1} = -\frac{f''(\xi_1)}{2f'(x_i)}(x^* - x_i)^2.$$

$$x^* - x_{i+1} = -\frac{f''(\xi_1)}{2\left[-f''(\xi_2)(x^* - x_i)\right]}(x^* - x_i)^2 = \frac{f''(\xi_1)}{2f''(\xi_2)}(x^* - x_i),$$

b. (3 points) Use (a) to argue that the method converges from a sufficiently good starting point, provided f'' is continuous and $f''(x^*) \neq 0$.

Let $M = |f''(x^*)| > 0$. Since f'' is continuous, there is a $\delta > 0$ such that $|f''(s) - f''(x^*)| \leq M/5$, $\forall s \in I = [x^* - \delta, x^* + \delta]$. On I, f'' is then nonzero and $\frac{4}{5}M \leq |f''(s)| \leq \frac{6}{5}M$, so for any $s, t \in I$, $\left|\frac{f''(s)}{2f''(t)}\right| \leq \frac{6/5}{2 \cdot 4/5} \rightarrow \left|\frac{f''(s)}{2f''(t)}\right| \leq \frac{3}{4}$
If $x_i \in I$, then $\xi_1, \xi_2 \in I$ and (a) gives $|x^* - x_{i+1}| \leq \frac{3}{4}|x^* - x_i|$, so $x_{i+1} \in I$ too. Starting from any $x_0 \in I$, by induction $|x^* - x_i| \leq (3/4)^i |x^* - x_0| \rightarrow 0$.

c. (3 points) Once x_i is close to x^*, by roughly what factor does the error shrink on each iteration? Compare this rate with bisection and with Newton–Raphson when $f'(x^*) \neq 0$.

As $x_i \rightarrow x^*$, $\xi_1, \xi_2 \rightarrow x^*$, so $\frac{x^* - x_{i+1}}{x^* - x_i} = \frac{f''(\xi_1)}{2f''(\xi_2)} \rightarrow \frac{f''(x^*)}{2f''(x^*)} = \frac{1}{2}$.
The error halves each iteration so we will converge. That is the same rate as bisection. This is slower than Newton–Raphson at a simple zero, where $|e_{i+1}| \approx C|e_i|^2$ and the number of correct digits roughly doubles each step.

d. (3 points) Perform five iterations of Newton–Raphson on $f(x) = (x-1)^2$ from $x_0 = 2$ and report the ratio of successive errors. When the zero is unknown, how could you tell from the iterates that a run is behaving this way?

Here $f'(x) = 2(x-1)$, so for $x_i \neq 1$,

$$x_{i+1} = x_i - \frac{(x_i - 1)^2}{2(x_i - 1)} = \frac{x_i + 1}{2}.$$

i	x_i	error $\lvert x_i - 1\rvert$	ratio $\lvert x_i - 1\rvert / \lvert x_{i-1} - 1\rvert$
0	2	1	–
1	1.5	0.5	0.5
2	1.25	0.25	0.5
3	1.125	0.125	0.5
4	1.0625	0.0625	0.5
5	1.03125	0.03125	0.5

Every ratio is exactly $\frac{1}{2}$. When the zero is unknown, we can watch the step sizes $|x_{i+1} - x_i|$ and if they shrink by a constant factor, the run is converging linearly $\rightarrow f'(x^*) = 0$.